

Exact-Oracle Post-Placement Refinement for Multi-Bit Flip-Flop Banking: Structural Rebanking without Cascades

Anonymous Author(s)

Abstract

Multi-bit flip-flop (MBFF) banking trades clock power and area against timing. Most flows drive it with proxy cost models, and the mispricing compounds: each committed merge distorts the cost of every later decision. On the ICCAD 2024 CAD Contest Problem B benchmark, we show that the root cause is the pricing error rather than the move class. Structural rebanking moves that cascade under a one-hop proxy model (up to +254% score) become strictly profitable once re-priced exactly under staged, monotone acceptance. We present a post-placement refinement framework built on an evaluator-exact, side-effect-free incremental timing oracle. The oracle prices every candidate move in closed loop with the official cost function: bit re-pairing, structural rebanking of $2 \times 2b \rightarrow 4b$ and $4 \times 1b \rightarrow 4b$, positional relocation and swaps, and bin-density eviction. Exact pricing enables (i) batch local search that safely commits an order of magnitude more moves per round than conservative conflict-free batching, (ii) monotone-accept structural rebanking with rollback-safe staging, and (iii) refinement-side pricing of the density penalty term. A single configuration beats the contest top-3 best entry on all seven benchmarks; ours is the first published method to do so. It reaches a composite ratio of 0.953, where the strongest previously published methods reach 0.979–0.991, with every solution verified legal by the official evaluator.

CCS Concepts

• Hardware → Physical design (EDA).

Keywords

multi-bit flip-flop, banking, incremental timing analysis, detailed placement, local search

1 Introduction

Multi-bit flip-flop (MBFF) banking merges single-bit registers into multi-bit trays that share internal clock inverters. It cuts clock dynamic power and cell area but costs placement freedom and timing slack. Post-placement banking spans power-driven merging under timing constraints [1], interval-graph formulations [21], and flop trays in commercial flows [13]. The ICCAD 2024 CAD Contest Problem B [22] turned this into a reproducible benchmark that optimizes banking and placement jointly under the four-term cost $\alpha \cdot TNS + \beta \cdot \text{Power} + \gamma \cdot \text{Area} + \lambda \cdot \text{Density}$ (TNS : total negative slack), scored by an official evaluator on seven industrial-scale cases under hard legality constraints.

Nearly every published flow prices banking’s committed decisions with a proxy timing model. The error does not average out: each committed merge moves pins and distorts the slack landscape that prices every later decision. We measured this compounding directly. Our baseline, a top-ranked open-source contest flow, carries a one-hop timing proxy that underestimates evaluator TNS by roughly 34% on the hardest case. Under that proxy, cascades of

mispriced merges of two 2-bit cells into one 4-bit cell ($2 \times 2b \rightarrow 4b$) regress the score by up to +254%, and a debank-and-rebank pass by +134% (Sec. 2). Structural rebanking after legalization then looks too dangerous a move class. We claim the opposite: with exact per-move pricing and staged, monotone acceptance, the same move class becomes strictly profitable. A controlled counterfactual (Sec. 5.3) puts the price of proxy quotes at up to 0.9% of final score.

We therefore rebuild the refinement stage around exact pricing. Upstream we keep the open-source baseline flow from debanking through detailed placement. We previously strengthened this front end under the same proxy pricing (Sec. 2); those enhancements are not claims of this paper. Everything downstream of detailed placement is the contribution here:

- **An incremental timing oracle.** Evaluator-exact and side-effect-free, it prices any candidate move against the official cost function without mutating placement state. Incremental and full recomputation agree exactly in every round, and side-effect-free deltas match a real apply-and-revert to floating-point precision (Sec. 3.4).
- **Exactly re-priced batch local search.** Batched local search normally commits only moves that touch disjoint timing cones, a safety filter that keeps about 9% of improving candidates per round (Sec. 4.1). Exact re-pricing at apply time makes the filter unnecessary and commits an order of magnitude more: a stale quote can only cause a rejection.
- **Monotone-accept structural rebanking with rollback staging.** The engine stages merges ($2 \times 2b \rightarrow 4b$, $4 \times 1b \rightarrow 4b$) outside the committed netlist, prices them exactly, commits only on strict improvement, and rolls back completely otherwise.
- **Density-term pricing combined with exact timing pricing.** A repair pass evicts flip-flops from violating bins. It accepts an eviction chain only if the exact timing plus density delta improves total cost, so refinement sees all four cost terms.

The rebuild also removed a silent restriction: a stale bookkeeping flag had been excluding 32% of one hidden case’s flip-flops from refinement altogether (Sec. 4).

Figure 2 summarizes the framework. One configuration, unchanged across all seven benchmarks, beats the contest top-3 best entry on every case; ours is the first published method to do so. The composite ratio against the contest top-3 is 0.953, versus 0.991 for the contest winner’s method under the same baseline; it self-reports 0.988 against first place in a two-page late-breaking result [14]. Ours is the first full-length method paper at this quality level on this benchmark. TIMBER [7] reports strong runtime and memory results on the same cases but is not evaluator-comparable (Sec. 5.2). Every solution we report passes the official evaluator and both legality checkers.

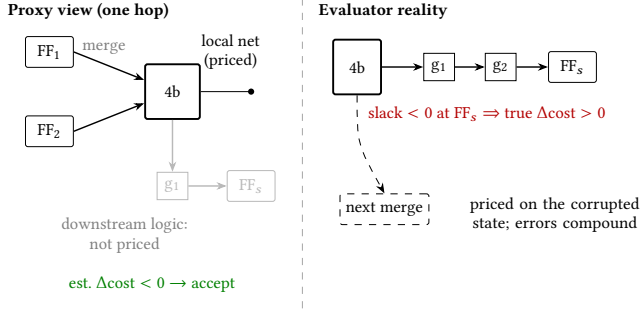


Figure 1: One merge, two prices: the one-hop proxy sees only the local net and accepts (left); the evaluator charges the downstream cone (right), and the next merge is priced on the corrupted state.

2 Background and Motivation

2.1 Problem B and the Official Evaluator

The ICCAD 2024 CAD Contest Problem B gives a legalized design: single- and multi-bit flip-flops, fixed combinational gates, and MBFF library variants differing in bit width, power, area, and pin geometry. A solution rebanks and replaces the flip-flops to minimize

$$S = \alpha \cdot TNS + \beta \cdot Power + \gamma \cdot Area + \lambda \cdot V, \quad (1)$$

where V counts bins whose utilization exceeds a per-testcase threshold; legality (no overlaps, all cells on rows inside the die) is a hard constraint [8, 22]. The weights $\alpha, \beta, \gamma, \lambda$ and a *DisplacementDelay* constant vary per testcase, so the dominant term shifts between TNS and power/area.

An official evaluator binary defines scoring, with timing semantics unlike conventional net models. Each arc’s delay is the *DisplacementDelay* constant times the two-point half-perimeter wirelength from driver pin to individual sink pin, with no shared net bounding box or Steiner topology. Each sink’s slack starts from its parse-time base value and moves by two deltas: the Q-pin delay change when the driving flip-flop changes cell type, and the wire-delay change of affected arcs. TNS sums negative D-pin slacks over all flip-flops. A refinement engine seeking monotone score descent must reproduce these semantics, since Eq. (1) is evaluated on them alone.

Our flow builds on a top-ranked open-source implementation of the contest reference flow (Fig. 2): debank MBFFs into single-bit registers, spread by slack-driven conjugate-gradient global placement, cluster by mean shift, bank, legalize onto rows, run detailed placement. We previously strengthened several stages: maximum-weight-matching pairing with proximity- and risk-weighted edge costs replaced greedy banking, and detailed placement gained cost-aware repairs. These enhancements keep the one-hop proxy pricing and are not claims of this paper; they make every refinement delta in Sec. 5 conservative, since refinement is measured on the strengthened front end. Everything from the incremental oracle onward (Secs. 3–4) is this paper’s contribution.

2.2 Compounding Mispricing under Proxy Cost Models

Refinement explores millions of candidate moves per benchmark, so per-candidate evaluator calls are infeasible; contest-style flows substitute an internal proxy. The baseline’s proxy is representative: it reads stored slacks and re-prices one hop of wiring around the moved cell. Fig. 1 shows why this cascades: the evaluator charges the merged cell through its downstream fanout, whose stored slacks the proxy never updates; each committed move thus corrupts the state that prices the next one. On the hardest benchmark, the compounded error leaves the one-hop model reporting final TNS roughly 34% below what the evaluator’s score decomposition implies.

Two prior structural rebanking attempts in the same codebase measured the cascade. One matched flip-flops into higher-bit cells, one debanked then rebanked mispriced groups; they degraded the score by +254% and +134% on a power-dominated hidden case. Both generated the move class we later exploit profitably: merging smaller flip-flops into 4-bit cells. The moves were sound and the one-hop prices were wrong. These cascades motivate the prediction, confirmed in Sec. 5, that exact pricing turns the same move class into strict descent.

2.3 Related Work

MBFF banking has a long post-placement lineage, from timing-constrained merging through flop-tray co-design to capacitated clustering at scale [1, 12, 13, 21]. Chen et al. [4] redistribute positive slack along register chains to enlarge feasible clustering regions before banking. Despite the shared term, theirs is a pre-clustering slack-budget transformation; we re-price every move exactly after placement.

On Problem B itself, the contest winner’s method appears only as a two-page late-breaking result [14] self-reporting a 0.988 average ratio against the first-place entry. The format precludes a full method description; to our knowledge no full-length paper has matched that level on this benchmark. Chen et al. [2] adopt a banking-then-refinement flow close to ours but price moves with proxy models. TIMBER [7] reports on the same testcases but is not evaluator-comparable (Sec. 5.2). Gong et al. [9] target MBFF timing and power at advanced nodes without a public evaluator.

Timing-driven placers attack the same objective [15, 16, 20] but optimize proxy timing signals without closing the loop with the scoring function. Incremental STA engines [10, 11] maintain full-netlist timing under general delay models; our oracle is narrower and cheap enough for inner-loop pricing. Global placers handle density analytically [3, 18]. No published work priced the contest’s bin-violation term $\lambda \cdot V$ in refinement together with exact timing; we price it per move alongside TNS .

3 An Evaluator-Exact Incremental Timing Oracle

3.1 Design Goal

The oracle prices every candidate move under the evaluator’s own timing model: per-sink displacement delay proportional to the two-point HPWL between driver and sink pins, under one ideal

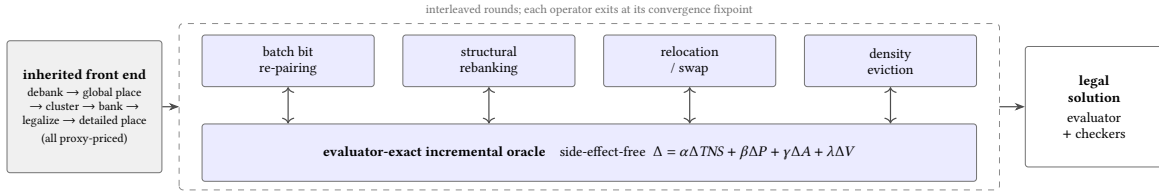


Figure 2: Framework: operators price every accept through one side-effect-free exact-delta interface; outputs are verified by the official evaluator and checkers.

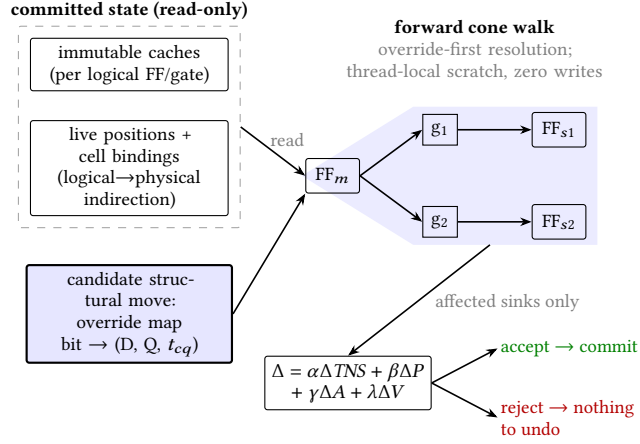


Figure 3: Side-effect-free delta evaluation: an override map plus a forward cone walk in thread-local scratch; a rejected candidate leaves nothing to undo.

clock [22]. It needs three properties at once. *Exactness*: the reported ΔTNS equals what the evaluator would measure. *Side-effect freedom*: pricing mutates no netlist or timing state, so thousands of candidates can be screened without apply-revert churn. *Locality*: cost scales with the affected timing cone, not the design.

Incremental STA engines meet the last two goals for full industrial timing [10, 11]. The contest model further permits closed-form per-sink recomputation at cone cost, which we exploit. The same simplicity explains why proxies are common. The inherited flow uses a one-hop slack approximation, the DATE 2026 flow a local critical-path wirelength proxy [2], and [14] leaves pricing unspecified. None closes the loop with the scoring function. On the hardest benchmark, the one-hop proxy underestimates TNS by roughly 34%; this mispricing makes structural rebanking cascade (Sec. 2). The oracle removes the model error entirely instead of tightening the approximation.

3.2 Timing State Keyed by Logical Objects

Banking creates and destroys physical cells, so any timing cache keyed by physical instances breaks on exactly the moves that need accurate pricing.

The oracle instead keys all topology caches by *immutable logical objects*: the single-bit flip-flops of the debanked netlist and the combinational gates. Banking never changes either set; a merge only remaps logical bits to a new (cell, slot) pair. Fanin and fanout

cones are computed once over these objects and survive every structural mutation. Mutable leaf attributes are never cached. A bit's D/Q pin coordinates and Q-pin (clock-to-Q) delay are read live through a logical→physical indirection: bit → cell and slot → library pin offset plus cell position. A structural move updates one indirection entry; no cache is rebuilt.

3.3 Three Evaluation Modes

Three entry points share one cone-propagation core.

Full pass. Recompute every sink slack from scratch in one parallel sweep. This is the validation ground truth, linear in design size.

Incremental cone recompute. After a move commits, repropagate only the cones reachable from the touched pins and fold the updates into the committed caches, keeping committed state exact at cone cost.

Side-effect-free delta. Price a hypothetical move without touching committed state (Fig. 3). The oracle builds a local *override map* holding each moved bit's hypothetical D/Q positions and Q-pin delay, collects the affected sinks, walks their forward cones override-first into thread-local scratch, and returns $\alpha \cdot \Delta TNS$; the scratch is discarded. Nothing writes committed state, so arbitrarily many candidates can be priced concurrently against it. The operators of Section 4 screen candidates in parallel through this mode and re-price serially at apply time.

3.4 Exactness Validation

Validation builds check two invariants continuously. After every refinement round, the incrementally maintained TNS total is compared against a full-pass recomputation; the difference is 0.000000 on every round of every benchmark. Each side-effect-free delta is also compared against the measured delta of applying and reverting that move; the maximum discrepancy is 7.4×10^{-13} , from floating-point summation order. The oracle is not a tighter estimate; it implements the declared scoring semantics to floating-point precision. Externally, the maintained totals track the official evaluator within 0.05% (median) at stage-boundary checkpoints on all seven cases; the maximum residual is 1.1% on the most TNS -weighted states. Controlled single-cell probes localize this residual to bounded-depth slack-state updates on committed moves, not to the delta semantics, which match the evaluator to 10^{-5} . Regardless, the evaluator binary itself scores every reported result (Sec. 5.2), so no claim here rests on the residual.

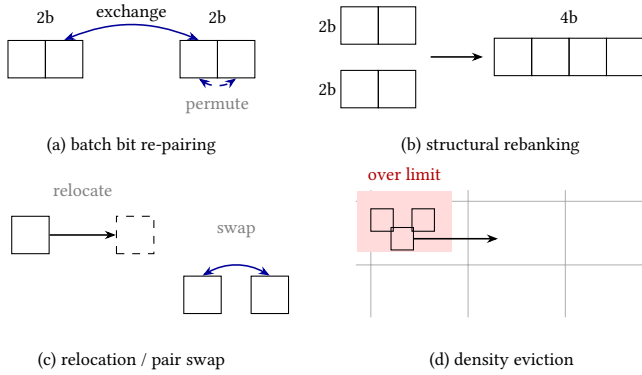


Figure 4: The four refinement moves: (a) inter-cell bit exchange and intra-cell slot permutation; (b) rebanking to higher-bit cells; (c) relocation and same-footprint swap; (d) eviction from over-utilized bins.

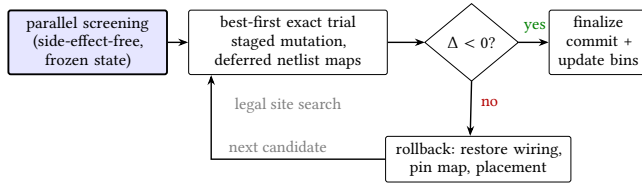


Figure 5: Staged commit lifecycle: screen on the frozen state, trial best-first, commit only on a negative exact delta; failed trials roll back completely.

3.5 From Bit Swaps to Structural Remaps

We first built the delta evaluator for same-cell bit swaps, where only pin positions change. The override map generalizes it at no extra cost. Remapping a *group* of bits to a new cell adds one field, the target cell's Q-pin delay override, because a new cell type also changes clock-to-Q delay. One mechanism therefore prices the whole move spectrum: slot swaps, relocations, and group-to-new-cell structural merges. The oracle-priced rebanking of Section 4 can thus screen structural candidates exactly before any netlist mutation.

Exactness is defined against the benchmark's declared cost model, but nothing in the framework depends on that model's simplicity. The operators of Section 4 see only one interface: a priced Δ for a hypothetical move. Under richer delay models, an incremental signoff-grade engine [10] can fill that interface, trading some pricing fidelity for generality.

4 Exactly Priced Refinement Operators

The oracle of Section 3 turns refinement into a pricing problem: it quotes any move's exact official-cost change before commit. Three move classes are priced exactly: bit re-pairing, structural rebanking, and bin-density eviction. Two positional operators, relocation and critical-pair swap, come from the reference flow. The framework of Section 4.4 interleaves all five; Figure 4 sketches these moves.

The three exactly-priced classes share one contract. The engine screens each candidate with a side-effect-free delta, re-prices it

exactly against the committed state at apply time, and accepts only if the full weighted delta $\alpha \Delta TNS + \beta \Delta P + \gamma \Delta A + \lambda \Delta V$ is negative. Committed sequences therefore decrease the official cost monotonically. The positional operators are narrower: they accept on the exact *TNS* delta at fixed power and area, without pricing the bin term, so *V* can drift during relocation. The closing density stage re-prices every remaining violation.

4.1 Batch Bit Re-Pairing

The finest-grained moves exchange bits between two multi-bit cells or permute bit-to-slot assignments within one cell (Fig. 4a). The search is batched best-improvement local search, in the spirit of multi-move-per-round scheduling methods [6] and batched placement refinement [17, 19]. Each round enumerates the best swap per cell in a candidate neighborhood, then commits many improving swaps together. Unlike dynasearch proper, it needs no independence structure between moves; each commit is re-priced exactly (below). Screening touches no shared mutable state, so threads price disjoint candidate lists in parallel against a frozen snapshot of the committed placement.

Exact pricing changes what a batch may contain. Conventional batched local search commits only conflict-disjoint candidates, because overlapping timing cones invalidate screened gains. Here each survivor is re-quoted against the committed state just before apply, so a stale gain can only cause a rejection, never a bad commit. The inherited flow's disjoint batching committed roughly 9% of improving candidates per round (152 of 1,703 on one hidden case). Dropping the disjointness filter commits every candidate still improving at apply time, an order-of-magnitude throughput gain at identical safety.

Intra-cell slot permutation complements inter-cell exchange. The evaluator prices each sink pin's displacement individually, so permuting which bit occupies which slot changes timing cost without moving the cell; the same delta evaluator prices these moves in the same batches.

Incremental engines converge to local optimality only over candidates they have seen. Periodic full rescans re-enumerate the whole move space and reseed the queue. Between rescans the engine drains cheap dirty-set candidates; each rescan certifies that no improving move was orphaned by earlier commits. The invariants of Sec. 3.4 check the engine's exactness continuously.

4.2 Oracle-Guided Structural Rebanking

Structural rebanking changes the banking solution after legalization, merging $2 \times 2b \rightarrow 4b$ and $4 \times 1b \rightarrow 4b$ (Fig. 4b). Candidates come from each cell's same-clock *k*-nearest neighbors, so enumeration stays linear in design size.

Each candidate passes through two phases. A parallel screening phase prices the merge with the group-move oracle at a centroid estimate, on the full objective $\alpha \Delta TNS + \beta \Delta P + \gamma \Delta A + \lambda \Delta V$. Power, area, and density deltas are exact from the cell library and bin table; the timing delta assumes the merged cell lands at the candidates' centroid. Screening is side-effect free, so all candidates are priced concurrently. An exact phase then processes survivors in best-first order of screened gain. Each merge is staged rather than applied directly (Fig. 5): mutations are deferred, a legal site is found for

the merged cell, and the oracle re-prices the move there; a negative exact delta commits the staged state, and rollback otherwise restores wiring, pin mapping, and placement. Acceptance is strictly monotone, with a patience cutoff.

The same move class cascades under one-hop proxy pricing (Sec. 2). Under exact pricing it descends strictly monotonically; on one hidden case each $2 \times 2b \rightarrow 4b$ merge alone is worth $-3,000$ weighted power-and-area units. The accept pricing and the staged, rollback-safe commits make the difference; Sec. 5.3 separates the two contributions experimentally.

4.3 Density-Term Pricing

The density penalty $\lambda \cdot V$ counts bins whose utilization exceeds the limit (Fig. 4d). Density is classically shaped during global placement [3, 18]. Prior refinement operators in this flow never priced the term, so none could trade timing against it. We price it directly alongside exact timing. For each violating bin, an eviction chain relocates flip-flops to legal sites outside it. The chain is priced as a unit, $\alpha \sum \Delta TNS + \lambda \Delta V$, and committed only if the total is negative; otherwise every eviction is reverted. Chain-level commit-or-revert matters: a violation clears only when the bin drops below the limit, so partial evictions pay timing cost for no penalty reduction.

4.4 Interleaved Anytime Framework

The operators run as interleaved rounds: relocation, critical-pair swap, bit re-pairing, and structural rebanking, repeated twice, with density repair closing the schedule. Both inherited positional operators stay as move generators; the oracle re-prices them. Timing-driven relocation moves a flip-flop to a legalized site near its slack-weighted driver centroid (Fig. 4c). It accepts only on exact TNS improvement at fixed power and area; the closing density stage handles the bin term. Critical-pair swap exchanges two same-footprint cells, leaving power, area, and legality unchanged, so acceptance reduces to the sign of the exact timing delta. Neither operator is new; the oracle of Section 3 now quotes their accepts instead of a one-hop proxy.

Interleaving pays because merges expose new re-pairing opportunities and vice versa. On one hidden case the interleaved schedule beat a single pass of the same operators by 185K weighted cost units. Each operator exits its round early once it converges, returning unused budget to the schedule. Structural commits are monotone in the official cost and the density stage closes the schedule, so the framework is anytime in practice. Interrupting it at any point yields a legal solution; in all recorded runs the official score descends monotonically across stage boundaries (Fig. 6).

One repair to the inherited flow determines the move space of all operators. The flow tested a per-cell boolean as a placement-liveness predicate for refinement candidates. Auditing its writers showed the flag is a work-queue marker for the banking stage: clustering clears it, only banking's commit paths set it, and cells placed by the final legalizer keep a stale value forever. On one hidden case the flag silently excluded 2,024 of 6,309 physical flip-flops (32%) from every refinement operator. The excluded set held all 2,000 two-bit cells, the entire two-bit exchange space of Section 4.1. We replace every flag test with membership in the placement map, the structure

the solution writer and legality checkers treat as ground truth. Refinement then covers the complete move space by construction.

5 Experimental Results

5.1 Setup

We evaluate on all seven ICCAD 2024 CAD Contest Problem B benchmarks [8, 22]: three released and four hidden cases spanning millions of instances and widely different weight profiles. All scores are from the official preliminary evaluator on the final solution file; every solution passed its legality check and the contest sanity and placement checkers. The flow is C++ with OpenMP on the strengthened open-source front end [5] of Section 2; refinement deltas are measured against that strengthened base, not the stock public flow.

All results use one unified configuration: a single recipe, zero per-case hyperparameters. The recipe was fixed via no-regression sweeps over the same seven cases, so like all published results on this benchmark it is selected in-sample. Unlike them, no knob differs between cases, and it beat every per-case-tuned configuration this flow had produced. We will release all seven solution files for independent checking. Experiments ran on a dual Intel Xeon E5-2630 v4 server (2.2 GHz, 40 logical cores, 94 GB RAM) with 8 threads. Operators run to convergence fixpoints, so results are reproducible: six of seven cases are byte-identical across all three repeats. On Hidden 4 one accept's gain sits at floating-point noise and flips between repeats, moving the final score by $2 \times 10^{-4} \%$ (std/mean).

5.2 Main Results

Table 1 compares against every evaluator-comparable published result: the top-3 per-case contest costs, the winner's method LBR (a two-page late-breaking result [14], self-reported average ratio 0.988 vs. first place), and the DATE 2026 flow [2], self-reported 0.2% better than LBR on average. Ours alone beats the top-3 best on every case and wins all seven head to head: vs. LBR by 2.0% on Case 2 and 20.9% on Hidden 2, vs. DATE'26 by 4.3% and 12.3%. The composite does not hinge on one case: the geometric mean is 0.949, and excluding Hidden 2 still yields 0.986, ahead of both methods recomputed on the same six cases (LBR 0.997, DATE'26 0.999). Hidden 2's margin is mechanistic: on this power-dominated case, proxy-priced flows leave $2 \times 2b \rightarrow 4b$ merges unexploited that exact pricing captures safely. TIMBER [7] also reports on these cases, but under its own min-max normalization and a baseline re-run whose bin-violation counts differ from the official contest standings; its scores cannot be placed in this table. Its strengths are runtime and memory rather than the contest cost metric.

5.3 Ablation

Table 2 enables the operators cumulatively on the three most cost-sensitive cases. The base row is the strengthened proxy-priced front end of Section 2 with refinement disabled, so deltas attribute to refinement only. Attribution tracks per-case cost structure. Exact batch re-pairing is the largest lever everywhere (-0.9% to -8.6%). Move-space repair (an audit finding, not an algorithmic contribution) and structural rebanking pay where residual cost sits in banking structure (Case 2, Hidden 2); rebanking alone gives -1.4%

Table 1: Official evaluator scores on all seven Problem B cases (lower is better). Top-3 costs re-evaluated in [2]; LBR scores as published (four significant digits) [14]. Ratios and composites (average per-case ratio) use the same top-3-best baseline. Baselines in print differ slightly (Case 2 first place: 748,000 in [14] vs. 744,231 in [2]); we use the stricter, so our margins are conservative.

Case	Top-3 best	LBR [14]	DATE'26 [2]	Ours	Δ vs top-3	T (s)
Case 1	739,235,861	738,800,000	736,602,486	734,766,838	−0.60%	199
Case 2	744,231	738,400	756,559	723,780	−2.75%	1,216
Case 3	727,971,140	728,800,000	727,757,372	726,236,872	−0.24%	2,582
Hidden 1	31,507,917	31,270,000	30,946,401	30,199,301	−4.15%	203
Hidden 2	13,408,414	12,760,000	11,510,067	10,097,053	−24.70%	2,202
Hidden 3	55,941,500	55,920,000	55,973,041	55,789,826	−0.27%	1,787
Hidden 4	728,497,353	726,900,000	727,798,594	726,154,906	−0.32%	2,619
Composite	1.000	0.991	0.979	0.953		

Table 2: Cumulative ablation, official evaluator score (lower is better).

Configuration	Case 2	Hidden 1	Hidden 2
Front end (no refinement)	761,159	30,615,486	11,614,120
+ exact batch re-pairing	740,251	30,336,791	10,619,091
+ move-space repair	733,903	30,335,019	10,356,851
+ structural rebanking	723,780	30,327,025	10,097,053
+ density-term pricing	723,780	30,199,301	10,097,053

and −2.5% respectively. Density pricing only acts where bins overflow: it moves Hidden 1 by −0.4% and is a verified no-op on Case 2 and Hidden 2, whose bin counts are already legal. No stage regresses any of the three cases.

A controlled counterfactual isolates pricing: we rerun the full flow unchanged except that the inherited one-hop proxy quotes each rebanking accept; candidates, staging, and operators are identical. Every case degrades: +0.15% on Case 2, +0.90% on Hidden 1, +0.41% on Hidden 2. Exact pricing therefore wins even inside an identical architecture. The staged monotone commit structure separately contains the damage that historically compounded under in-flow proxy merging (Sec. 2); those cascades required losing both safeguards.

5.4 Anytime Behavior

Figure 6 plots official evaluator score against refinement wall-clock for the two cases with the most headroom; each point is one evaluator run at a stage boundary, and the descent is monotone across all 63 checkpoints on all seven cases.

5.5 Runtime

Table 1 lists per-case wall times; the full suite takes about three hours on one server. Cases that converge fast finish in under four minutes, and TNS-bound cases spend where descent continues. Bit re-pairing dominates (66–96% of refinement time); rebanking takes 4–108 s per case; density repair finishes in under 0.1 s everywhere and is a no-op on the three cases without overfilled bins.

The runtime–quality trade can be quoted at any budget: a 1–2.5 minute refinement cap (median 120 s wall) yields composite 0.966, a 3–8 minute class 0.958, and full convergence 0.953. Every

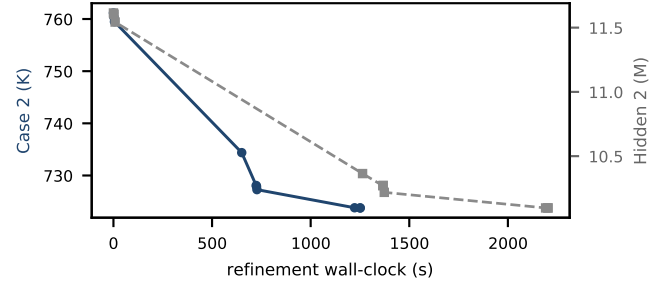


Figure 6: Official evaluator score versus refinement wall-clock. Case 2 (left axis, solid) and Hidden 2 (right axis, dashed).

point beats the best published composite (0.979). At LBR’s runtime class ours is already 2.5 points ahead; the two-minute tier meets contest-style budgets by construction.

The base flow completes in 21–36 s, comparable to the contest entries; the extra budget buys millions of exactly-priced moves that faster flows never attempt (on Hidden 2, screening prices about 4,000 structural candidates per second at 8 threads, and one exact trial-apply costs ~12 ms). For reference, DATE 2026 reaches 0.979 in ~3 s per case (2.5 GHz i7 [2]), LBR 0.991 in up to 77 s (3 GHz EPYC [14]), and ours 0.953 in minutes on an older 2.2 GHz server.

6 Conclusion

We presented a post-placement MBFF refinement framework whose accepts are priced by a side-effect-free incremental timing oracle. Exact pricing turns structural rebanking from a cascading liability into strictly monotone improvement and raises batch commit throughput by an order of magnitude. The density penalty obeys the same accept rule as timing. One configuration beat the best contest top-3 entry on all seven benchmarks: composite ratio 0.953, against 0.979–0.991 for the best published methods [2, 14]. Limitations: evidence covers one benchmark family and its cost function, the ideal-clock model excludes clock networks, and tight deadlines would reintroduce truncation variance; future work extends the discipline to debank/eject moves and richer bit widths.

References

- [1] Ya-Ting Chang, Chih-Cheng Hsu, Mark Po-Hung Lin, Yao-Wen Tsai, and Sheng-Fong Chen. 2010. Post-placement Power Optimization with Multi-Bit Flip-Flops. In *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. doi:10.1109/ICCAD.2010.5654155
- [2] Huan-Yuan Chen, Yu-Ruei Lin, Mark Po-Hung Lin, and Hung-Ming Chen. 2026. Optimizing Multibit Flip-Flop Banking via Agile In-Placement PPA Co-Optimization. In *Proc. Design, Automation and Test in Europe (DATE)*. doi:10.23919/DATE69613.2026.11539424
- [3] Tung-Chieh Chen, Zhe-Wei Jiang, Tien-Chang Hsu, Hsin-Chen Chen, and Yao-Wen Chang. 2008. NTUplace3: An Analytical Placer for Large-Scale Mixed-Size Designs with Preplaced Blocks and Density Constraints. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 27, 7 (2008), 1228–1240. doi:10.1109/TCAD.2008.923063
- [4] Yen-Yu Chen, Hao-Yu Wu, Iris Hui-Ru Jiang, Cheng-Hong Tsai, and Chien-Cheng Wu. 2024. Slack Redistributed Register Clustering with Mixed-Driving Strength Multi-bit Flip-Flops. In *Proc. International Symposium on Physical Design (ISPD)*. doi:10.1145/3626184.3633327
- [5] coherent17. 2024. 2024-ICCAD-Problem-B: open-source contest flow. <https://github.com/coherent17/2024-ICCAD-Problem-B>. Third-party baseline implementation.
- [6] Richard K. Congram, Chris N. Potts, and Steef L. van de Velde. 2002. An Iterated Dynasearch Algorithm for the Single-Machine Total Weighted Tardiness Scheduling Problem. *INFORMS Journal on Computing* 14, 1 (2002), 52–67. doi:10.1287/ijoc.14.1.52.7712
- [7] Aditya Das Sarma, Shui Jiang, Wan-Luan Lee, Tsung-Yi Ho, and Tsung-Wei Huang. 2026. TIMBER: A Fast Algorithm for Timing and Power Optimization using Multi-bit Flip-flops. In *Proc. Asia and South Pacific Design Automation Conference (ASP-DAC)*. doi:10.1109/ASP-DAC66049.2026.11420570
- [8] Shao-Yun Fang, Yi-Yu Liu, Chung-Kuan Cheng, and Tsun-Ming Tseng. 2024. Overview of 2024 CAD Contest at ICCAD. In *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. doi:10.1145/3676536.3689908
- [9] Tingxuan Gong, Wenxu Ruan, Dongwei Tan, Zhenhong He, Zhifeng Lin, and Jianli Chen. 2025. Multi-Bit Flip-Flop Based Timing and Power Optimization under Advanced Technology Nodes. In *Proc. IEEE International Symposium on Circuits and Systems (ISCAS)*. doi:10.1109/ISCAS56072.2025.11044076
- [10] Tsung-Wei Huang, Guannan Guo, Chun-Xun Lin, and Martin D. F. Wong. 2021. OpenTimer v2: A New Parallel Incremental Timing Analysis Engine. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 40, 4 (2021), 776–789. doi:10.1109/TCAD.2020.3007319
- [11] Tsung-Wei Huang and Martin D. F. Wong. 2015. OpenTimer: A High-Performance Timing Analysis Tool. In *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. doi:10.1109/ICCAD.2015.7372666
- [12] Andrew B. Kahng, Bodhisatta Kundu, and Dooseok Thumathy. 2024. Scalable Flip-Flop Clustering Using Divide and Conquer for Capacitated K-Means. In *Proc. Great Lakes Symposium on VLSI (GLSVLSI)*. doi:10.1145/3649476.3658744
- [13] Andrew B. Kahng, Jiajia Li, and Lutong Wang. 2016. Improved Flop Tray-Based Design Implementation for Power Reduction. In *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. doi:10.1145/2966986.2967047
- [14] Cheng-Yen Li, Chuan-Chi Su, Zheng-Wei Chen, Shao-Hsiang Chen, and Yao-Wen Chang. 2025. Late Breaking Results: Multi-Objective Multi-Bit Flip-Flop Placement Considering Pre-Placed Cells. In *Proc. Design Automation Conference (DAC)*. doi:10.1109/DAC63849.2025.11133043
- [15] Peiyu Liao, Siting Liu, Zhitang Chen, Wenlong Lv, Yibo Lin, and Bei Yu. 2022. DREAMPlace 4.0: Timing-driven Global Placement with Momentum-based Net Weighting. In *Proc. Design, Automation and Test in Europe (DATE)*. doi:10.23919/DATE54114.2022.9774725
- [16] Dhoni Lim and Heechun Park. 2025. Timing-Driven Detailed Placement with Unsupervised Graph Learning. In *Proc. Design, Automation and Test in Europe (DATE)*. doi:10.23919/DATE64628.2025.10992920
- [17] Yibo Lin, Wuxi Li, Jiaqi Gu, Haoxing Ren, Bruce Khailany, and David Z. Pan. 2020. ABCDPlace: Accelerated Batch-Based Concurrent Detailed Placement on Multithreaded CPUs and GPUs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 12 (2020), 5083–5096. doi:10.1109/TCAD.2020.2971531
- [18] Jingwei Lu, Pengwen Chen, Chin-Chih Chang, Lu Sha, Dennis Jen-Hsin Huang, Chin-Chi Teng, and Chung-Kuan Cheng. 2015. ePlace: Electrostatics-Based Placement Using Fast Fourier Transform and Nesterov's Method. *ACM Transactions on Design Automation of Electronic Systems* 20, 2 (2015). doi:10.1145/2699873
- [19] Min Pan, Natarajan Viswanathan, and Chris Chu. 2005. An Efficient and Effective Detailed Placement Algorithm. In *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 48–55. doi:10.1109/ICCAD.2005.1560039
- [20] Crisithian Roman-Vicharra, Jiang Chen, and Jiang Hu. 2024. Flip-Flop Centric Incremental Placement for Simultaneous Timing and Clock Network Power Optimization. In *Proc. ACM/IEEE Workshop on Machine Learning for CAD (MLCAD)*. doi:10.1145/3670474.3685949
- [21] Yaw-Tzong Shyu, Jai-Ming Lin, Chun-Po Huang, Cheng-Wu Lin, Ying-Zhih Lin, and Soon-Jyh Chang. 2013. Effective and Efficient Approach for Power Reduction by Using Multi-Bit Flip-Flops. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 21, 4 (2013), 624–635. doi:10.1109/TVLSI.2012.2190535
- [22] Sheng-Wei Yang, Jih-Wei Hsu, Ting-Wei Lee, Tzu-Hsuan Chen, and Cindy Chin-Fang Shen. 2024. 2024 ICCAD CAD Contest Problem B: Power and Timing Optimization Using Multibit Flip-Flop. In *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. doi:10.1145/3676536.3689911 Invited contest paper.